3D PROGRAMMING – FINAL PROJECT 2026

BUILD A COMPLETE 3D GAME

Format	Team Project
Team Size	3–4 students/team
Engine	Unity, Godot 4, or Unreal Engine 5
Total Score	10.0 points

1. Final Project Objectives

The Final Project is the capstone assessment for the 3D Programming course. Each team must develop a playable 3D game with a clear objective and demonstrate the ability to integrate knowledge and skills learned throughout the semester into a complete product.

By completing the project, students should demonstrate the ability to:

- Design and implement a 3D game with a clear game loop.
- Apply concepts related to Transform, Camera, Lighting, Materials, Collision, and Physics.
- Implement player control, interaction, gameplay mechanics, and game states.
- Organize source code/scripts/Blueprints appropriately and integrate multiple game systems.
- Package, demonstrate, and explain the important technical components of the project.

2. Project Scope

Each team must propose its own 3D game idea. Teams may choose one of the following genres or propose another suitable genre:

- First-Person Shooter (FPS), Third-Person Adventure, Action, Survival, or Horror.
- Endless Runner, Platformer, Racing, or Sports Game.
- Tower Defense, Puzzle/Escape Room, Stealth, or Exploration Game.
- Multiplayer Arena or another 3D game approved by the lecturer.

Students are not required to create all 3D models themselves. Legally usable assets from external libraries are allowed, but the main assessment focuses on gameplay, programming, system integration, and the overall completeness of the game.

3. Mandatory Requirements

The game must satisfy all of the following minimum requirements.

3.1. Player System

The game must include a main character or controllable object. At minimum, it must provide movement and a camera appropriate to the selected game genre. Additional actions may include jump, attack, shoot, dash, drive, or interact.

3.2. Gameplay Objective

The game must have a clear objective so that the player understands what to do. Examples include completing a level, collecting items, surviving, defeating enemies, solving a puzzle, or finishing a race.

3.3. Challenge System

The game must include at least one source of challenge, such as enemies, obstacles, traps, a time limit, puzzles, racing opponents, limited health/resources, or an equivalent mechanic.

3.4. Game Progression

The gameplay must include progression, such as increasing difficulty, multiple waves/levels, checkpoints, missions, score progression, unlocks, a boss, or another suitable progression mechanic.

3.5. Game States

At minimum, the game must include Start/Ready, Playing, Win or Game Over, and Restart states. A Main Menu and Pause state are recommended.

3.6. User Interface

The game must include a UI that displays information needed for gameplay, such as Health, Score, Timer, Ammo, Collected Items, Mission, Level, and Win/Game Over status.

4. Technical Requirements

The project must use at least 4 different game programming/3D techniques. These may include:

- Collision / Trigger; Rigidbody / Physics; Raycast.
- Animation; Camera control; Lighting; Particle System; Audio.
- Object spawning; Object pooling; procedural/random generation.
- State Machine; Enemy AI; Navigation / NavMesh.
- Save/Load; High Score; Inventory; Weapon System; Quest/Mission System.
- Networking/Multiplayer or another advanced technique appropriate to the project.

5. Advanced Feature

Each team must implement at least 1 Advanced Feature that has a meaningful role in the gameplay. Examples include:

- Enemy AI with Patrol/Chase/Attack or a Behavior Tree/State Machine.
- Procedural generation or dynamic difficulty.
- Boss fight, inventory, weapon switching, skill system, or quest system.

- Vehicle physics, advanced physics puzzles, multiplayer/networking, save/load, or an equivalent advanced system.

6. Independent Development and Project Growth

The Final Project must not be an unchanged copy of a tutorial or a previous class exercise. Teams may reuse knowledge, code, assets, or techniques from earlier work, but the final game must demonstrate significant further development.

Each project must include at least 2 features or design decisions independently developed or substantially adapted by the team to fit its own game.

7. External Resources and AI Tools

Students may use external resources and supporting tools, including the Unity Asset Store, Fab/Unreal Marketplace, Godot Asset Library, Mixamo, Sketchfab, FreeSound, GitHub, YouTube tutorials, official documentation, and AI tools.

However, teams must comply with the following requirements:

- Clearly declare important assets, tutorials, source code, and AI tools used.
- Understand and explain the important scripts/Blueprints/GDScript used in the project.
- Be able to change parameters, modify, or debug part of the functionality when requested during the demo.
- Do not submit another person's complete project as the team's own work.

8. Individual Contribution

Each member must clearly state their main contribution in the README or submission documentation. Example:

Member	Main Contribution
Student A	Player movement + combat
Student B	Enemy AI + navigation
Student C	Level design + spawning
Student D	UI + audio + game states

9. Minimum Requirements for a Complete Project

- A controllable player and a functional 3D environment.
- A clear gameplay objective and challenge.
- Win/Game Over functionality and the ability to Restart.
- A UI that supports gameplay.
- At least 4 game programming/technical features.
- At least 1 Advanced Feature.
- The project can be demonstrated and played during the assessment.

A project that only contains an attractive 3D scene and a movable character, but no game loop, objective, or challenge, will not be considered a complete Final Game.

158A Le Loi, Danang, Vietnam
www.vnuk.edu.vn | contact@vnuk.edu.vn
Phone: +84 (236) 3 64 65 77

10. Deliverables

10.1. Source Code / Project

Submit either a GitHub Repository or a project file (.zip). The source must be clearly organized and must be possible to open and run.

10.2. Playable Build

Submit a playable build when the engine and project allow it (for example, a Windows executable, WebGL build, Linux build, or packaged game). If technical limitations prevent a build, the team must be able to run and demonstrate the project directly in the editor.

10.3. Demo Video

Duration: 3–5 minutes. The video must show at least:

- A brief introduction to the game and its objective.
- Main gameplay and important mechanics.
- The Advanced Feature.
- Win/Game Over or the final gameplay result.

10.4. README

The README must include: Game Title; team members; game concept; controls; gameplay objective; main features; advanced feature; engine/technology; run instructions; asset/resource sources; AI tools used; and a contribution table for each member.

10.5. Final Presentation & Live Demo

Each team will have approximately 10–12 minutes for the presentation and live demo. Suggested structure:

- 2 minutes: Game concept and objective.
- 5 minutes: Live gameplay.
- 3 minutes: Main technical explanation.
- 2 minutes: Q&A / individual understanding check.

11. Submission Schedule

Specific dates and times will be announced or updated on the LMS. Teams are responsible for following the published deadlines. The official submission schedule is:

Milestone	Activity	Submission Requirement	Date/Time
Milestone 1	Project Registration	Game title, genre, engine, members, brief description	See LMS
Milestone 2	Progress Check	Demo of the current build and main mechanics	See LMS
Milestone 3	Final Submission	Source/Repository, Build, Video, README	See LMS
Milestone 4	Final Demo	Presentation + Live Demo + Q&A	According to class schedule

158A Le Loi, Danang, Vietnam
www.vnuk.edu.vn | contact@vnuk.edu.vn
Phone: +84 (236) 3 64 65 77

12. Assessment Rubric – 10.0 points

Criterion	Description	Points
1. Game Concept & Objective	Appropriate concept; the game objective is clear and the player understands what to do.	0.5
2. Core Gameplay	Player control, core gameplay, and the game loop work well; the game is genuinely playable.	2.0
3. Game Mechanics & Progression	Mechanics are meaningfully connected; the game includes suitable challenge and progression/level/wave/mission/score systems.	1.5
4. Technical Implementation	3D/game programming techniques are applied and integrated effectively; code/scripts/Blueprints are organized.	2.0
5. Advanced Feature	At least 1 advanced feature works in practice and has a meaningful role in gameplay.	1.0
6. Game & Level Design	Level/environment layout is appropriate; gameplay has suitable flow, difficulty, and player experience.	1.0
7. UI / Audio / Visual Polish	UI is clear; audio, lighting, animation/effects support the game experience.	0.5
8. Stability & Playability	The game is stable, has no major errors, can be completed/restarted, and can be demonstrated.	0.5
9. Presentation & Technical Explanation	The demo is clear; the team can explain the technical implementation, resources used, and individual contributions.	1.0

Total: 10.0 points

13. Assessment and Demo Regulations

- The lecturer may ask individual members to explain a script/Blueprint/GDScript or a system implemented by the team.
- The lecturer may ask the team to change a gameplay parameter or make a small modification to check understanding of the project.
- If a declared feature does not work during the demo, that feature may not receive credit.
- Individual marks may be adjusted when there is a significant difference in contribution or ability to explain the project.
- All important resources, tutorials, and AI tools must be honestly declared in the README.

14. Pre-Submission Checklist

☐ Player control works ☐ Clear objective ☐ Challenge ☐ Progression
☐ Win/Game Over + Restart ☐ UI ☐ ≥ 4 technical features ☐ ≥ 1 advanced feature
☐ Source/Repository ☐ Build ☐ Demo video ☐ README ☐ Contribution + resource declaration

158A Le Loi, Danang, Vietnam
www.vnuk.edu.vn | contact@vnuk.edu.vn
Phone: +84 (236) 3 64 65 77